

CS3014 - Estructuras de Datos Avanzadas

Dynamic Optimality II

Semana 6

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

15 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión



- ▶ Los tres casos de SPLAY: Zig, Zig-Zig, Zig-Zag

Objetivos de la sesión



- ▶ Los tres casos de SPLAY: Zig, Zig-Zig, Zig-Zag
- ▶ BUSCAR, INSERTAR y ELIMINAR, contruidos sobre SPLAY

Objetivos de la sesión



- ▶ Los tres casos de SPLAY: Zig, Zig-Zig, Zig-Zag
- ▶ BUSCAR, INSERTAR y ELIMINAR, contruidos sobre SPLAY
- ▶ El método del potencial, adaptado a Splay Trees

Objetivos de la sesión



- ▶ Los tres casos de SPLAY: Zig, Zig-Zig, Zig-Zag
- ▶ BUSCAR, INSERTAR y ELIMINAR, contruidos sobre SPLAY
- ▶ El método del potencial, adaptado a Splay Trees
- ▶ El Lema de Acceso, con su prueba completa

Objetivos de la sesión



- ▶ Los tres casos de SPLAY: Zig, Zig-Zig, Zig-Zag
- ▶ BUSCAR, INSERTAR y ELIMINAR, contruidos sobre SPLAY
- ▶ El método del potencial, adaptado a Splay Trees
- ▶ El Lema de Acceso, con su prueba completa
- ▶ Consecuencias: $O(\lg n)$ amortizado, y algo más fuerte

Motivación



Dónde quedamos

Dijimos que un Splay Tree cumple varias propiedades fuertes — pero nunca vimos **cómo** funciona SPLAY exactamente, **cómo** se construyen las operaciones básicas sobre él, ni **por qué** su costo amortizado es $O(\lg n)$.

Motivación



Dónde quedamos

Dijimos que un Splay Tree cumple varias propiedades fuertes — pero nunca vimos **cómo** funciona SPLAY exactamente, **cómo** se construyen las operaciones básicas sobre él, ni **por qué** su costo amortizado es $O(\lg n)$.

¡Piénsalo!

Cada búsqueda mueve x hasta la raíz — en el peor caso, eso son $\Theta(n)$ rotaciones para **una** búsqueda. ¿Cómo puede ser $O(\lg n)$ **amortizado** si una operación individual puede costar tanto?

Motivación



Dónde quedamos

Dijimos que un Splay Tree cumple varias propiedades fuertes — pero nunca vimos **cómo** funciona SPLAY exactamente, **cómo** se construyen las operaciones básicas sobre él, ni **por qué** su costo amortizado es $O(\lg n)$.

¡Piénsalo!

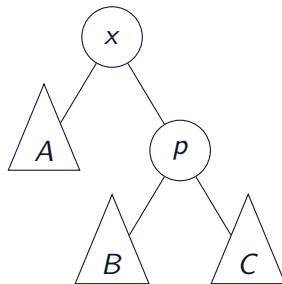
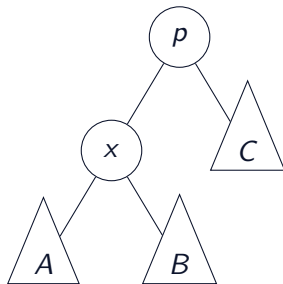
Cada búsqueda mueve x hasta la raíz — en el peor caso, eso son $\Theta(n)$ rotaciones para **una** búsqueda. ¿Cómo puede ser $O(\lg n)$ **amortizado** si una operación individual puede costar tanto?

La respuesta: análisis amortizado real

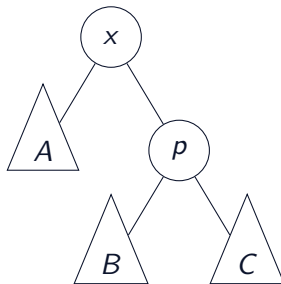
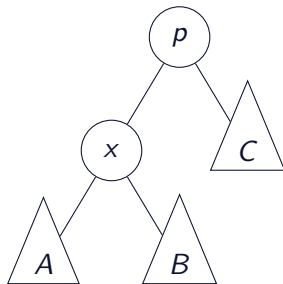
Necesitamos una función de **potencial** que capture cuánto “desorden” hay guardado en el árbol — y mostrar que, aunque una operación sea cara, el potencial baja lo suficiente para compensarlo.

Los casos de rotación

Zig: el caso base

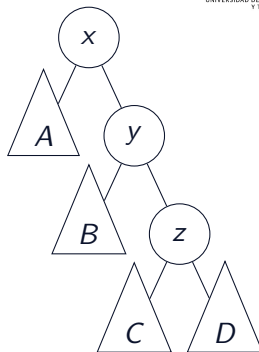
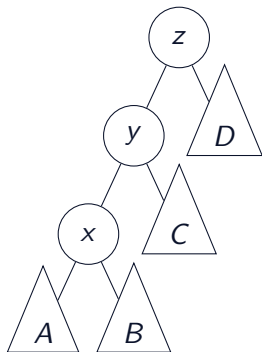


Zig: el caso base

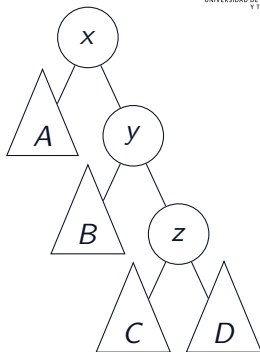
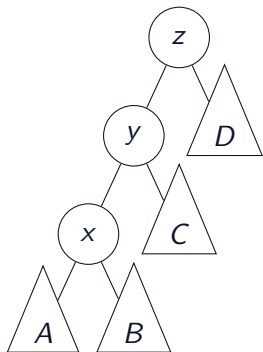


x es hijo de la **raíz**: una rotación simple, y listo — el caso que termina el SPLAY.

Zig-Zig: misma dirección



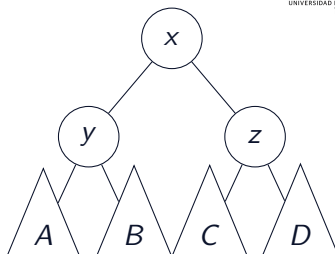
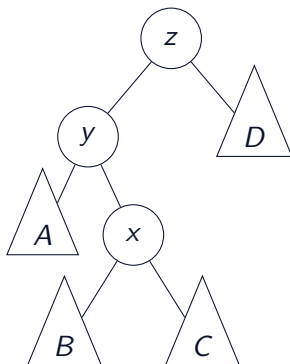
Zig-Zig: misma dirección



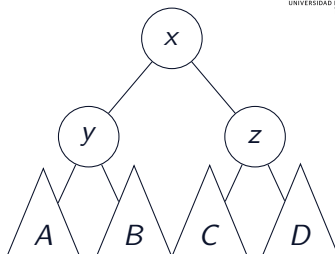
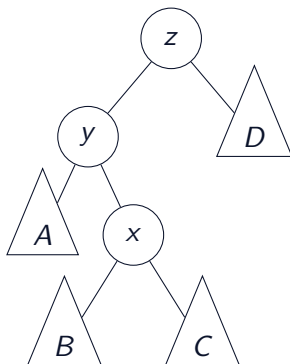
¡Piénsalo!

x y y son **ambos** hijos izquierdos (o ambos derechos) de sus padres. ¿Por qué no basta con rotar x con y , y luego x con z (dos rotaciones “ingenuas”)?

Zig-Zag: direcciones opuestas



Zig-Zag: direcciones opuestas



x es hijo izquierdo de y , pero y es hijo derecho de z (o simétrico): rotar x dos veces en direcciones **opuestas** — x termina como raíz del subárbol, con y y z como sus dos hijos.

La rotación primitiva: Rotar



Una sola idea, reutilizada tres veces

Los tres casos (Zig, Zig-Zig, Zig-Zag) se arman todos a partir de **una** operación primitiva: rotar un nodo v con su padre, para que v suba un nivel.

Zig, Zig-Zig y Zig-Zag, en términos de Rotar



¡Piénsalo!

Ya vimos (Observación de la diapositiva de Zig-Zig) que el **orden** de las rotaciones importa: primero y con z , luego x con y — nunca al revés. ¿Cómo se traduce eso a llamadas de ROTAR?

Zig, Zig-Zig y Zig-Zag, en términos de Rotar



¡Piénsalo!

Ya vimos (Observación de la diapositiva de Zig-Zig) que el **orden** de las rotaciones importa: primero y con z , luego x con y — nunca al revés. ¿Cómo se traduce eso a llamadas de ROTAR?

Caso	En términos de Rotar
Zig	$\text{ROTAR}(x)$
Zig-Zig	$\text{ROTAR}(\text{padre}(x)); \text{ROTAR}(x)$
Zig-Zag	$\text{ROTAR}(x); \text{ROTAR}(x)$

Zig, Zig-Zig y Zig-Zag, en términos de Rotar

¡Piénsalo!

Ya vimos (Observación de la diapositiva de Zig-Zig) que el **orden** de las rotaciones importa: primero y con z , luego x con y — nunca al revés. ¿Cómo se traduce eso a llamadas de ROTAR?

Caso	En términos de Rotar
Zig	$\text{ROTAR}(x)$
Zig-Zig	$\text{ROTAR}(\text{padre}(x)); \text{ROTAR}(x)$
Zig-Zag	$\text{ROTAR}(x); \text{ROTAR}(x)$

En Zig-Zig, la **primera** llamada es sobre el **padre** de x (rota y con z); en Zig-Zag, ambas llamadas son sobre x mismo (rota x con y , y luego el nuevo x — ya en el lugar de y — con z). El mismo primitivo, combinado distinto, da los tres casos.

Splay, completo

Algoritmo 1: SPLAY(x)

mientras $x \neq \text{raiz}$ **hacer**

$p \leftarrow \text{padre}(x)$

si $p = \text{raiz}$ **entonces**

 ROTAR(x)

// Zig

en otro caso

$g \leftarrow \text{padre}(p)$

si $(x = p.\text{izq}) = (p = g.\text{izq})$ **entonces**

 ROTAR(p); ROTAR(x)

// Zig-Zig

en otro caso

 ROTAR(x); ROTAR(x)

// Zig-Zag

Splay, completo

Algoritmo 2: $\text{SPLAY}(x)$

mientras $x \neq \text{raiz}$ **hacer**

$p \leftarrow \text{padre}(x)$

si $p = \text{raiz}$ **entonces**

 ROTAR(x)

// Zig

en otro caso

$g \leftarrow \text{padre}(p)$

si $(x = p.\text{izq}) = (p = g.\text{izq})$ **entonces**

 ROTAR(p); ROTAR(x)

// Zig-Zig

en otro caso

 ROTAR(x); ROTAR(x)

// Zig-Zag

Cada búsqueda de x termina con $\text{SPLAY}(x)$: el costo real es proporcional a la profundidad de x — hasta $\Theta(n)$ en el peor caso individual.

Operaciones básicas

¿Qué necesitamos, además de Splay?



El TDA de un árbol de búsqueda

Todo árbol de búsqueda necesita, como mínimo, **Buscar, Insertar y Eliminar**. Ya tenemos SPLAY — ¿alcanza con eso?

¿Qué necesitamos, además de Splay?



El TDA de un árbol de búsqueda

Todo árbol de búsqueda necesita, como mínimo, **Buscar, Insertar y Eliminar**. Ya tenemos SPLAY — ¿alcanza con eso?

¡Piénsalo!

¿Podemos construir las tres operaciones usando **solo** una función de búsqueda y SPLAY, sin ningún recorrido extra del árbol?

¿Qué necesitamos, además de Splay?



El TDA de un árbol de búsqueda

Todo árbol de búsqueda necesita, como mínimo, **Buscar**, **Insertar** y **Eliminar**. Ya tenemos **SPLAY** — ¿alcanza con eso?

¡Piénsalo!

¿Podemos construir las tres operaciones usando **solo** una función de búsqueda y **SPLAY**, sin ningún recorrido extra del árbol?

Sí — y de regalo, dos operaciones más

Vamos a construir **BUSCAR** (que ya usa **SPLAY**), y a partir de ella **SEPARAR** y **UNIR** — dos primitivos casi gratuitos que hacen que **INSERTAR** y **ELIMINAR** salgan en pocas líneas.

Buscar: el ingrediente base

Algoritmo 3: BUSCAR(k)

```
 $v \leftarrow \text{raiz}, \text{ultimo} \leftarrow \text{NULO}$   
mientras  $v \neq \text{NULO}$  hacer  
     $\text{ultimo} \leftarrow v$   
    si  $k = v.\text{valor}$  entonces  
        SPLAY( $v$ )  
        devolver verdadero  
    en otro caso  
        si  $k < v.\text{valor}$  entonces  
             $v \leftarrow v.\text{izq}$   
        en otro caso  
             $v \leftarrow v.\text{der}$   
si  $\text{ultimo} \neq \text{NULO}$  entonces  
    SPLAY( $\text{ultimo}$ )  
devolver falso
```

Buscar: el ingrediente base



Si k no está, **BUSCAR** splaya el **último** nodo visitado — el predecesor o sucesor de k . Esta es, literalmente, la operación de “acceso” que vamos a analizar con el Lema de Acceso.

Separar: partir el árbol en dos



Algoritmo 4: SEPARAR(k)

```
// Precondición: ya se ejecutó BUSCAR( $k$ )
si  $raiz = NULO$  entonces
    | devolver ( $NULO, NULO$ )
 $r \leftarrow raiz$ 
si  $r.valor \leq k$  entonces
    |  $l \leftarrow r, D \leftarrow r.der, r.der \leftarrow NULO$ 
en otro caso
    |  $D \leftarrow r, l \leftarrow r.izq, r.izq \leftarrow NULO$ 
devolver ( $l, D$ )
```

Separar: por qué es $O(1)$ adicional



¡Piénsalo!

Tras $\text{BUSCAR}(k)$, la raíz r ya cumple: uno de sus dos subárboles es todo $\leq k$ y el otro todo $> k$. ¿Cuántas comparaciones más hacen falta para partirlo?

Separar: por qué es $O(1)$ adicional



¡Piénsalo!

Tras $\text{BUSCAR}(k)$, la raíz r ya cumple: uno de sus dos subárboles es todo $\leq k$ y el otro todo $> k$. ¿Cuántas comparaciones más hacen falta para partirlo?

Ninguna: solo hay que mirar el valor de r y cortar **un** puntero. Todo el trabajo de “recorrer” ya lo pagó BUSCAR — SEPARAR es $O(1)$ puro encima de él.

Unir: pegar dos árboles



Precondición: todo valor en A es menor que todo valor en B ; $m = \text{máximo de } A$.

Unir: pegar dos árboles



Precondición: todo valor en A es menor que todo valor en B ; $m = \text{máximo de } A$.

Algoritmo 6: $\text{UNIR}(A, B)$

si $A = \text{NULO}$ **entonces**

 | **devolver** B

si $B = \text{NULO}$ **entonces**

 | **devolver** A

$v \leftarrow A$

mientras $v.\text{der} \neq \text{NULO}$ **hacer**

 | $v \leftarrow v.\text{der}$

$\text{SPLAY}(v)$

// $v = m$, ahora raíz de A

$v.\text{der} \leftarrow B$

devolver v

Insertar, en pocas líneas



Algoritmo 7: INSERTAR(k)

si BUSCAR(k) entonces

 devolver

// k ya estaba en el árbol

en otro caso

$(I, D) \leftarrow \text{SEPARAR}(k)$

$x \leftarrow$ nuevo nodo con valor k

$x.\text{izq} \leftarrow I$, $x.\text{der} \leftarrow D$, $\text{raiz} \leftarrow x$

Insertar, en pocas líneas



Algoritmo 8: INSERTAR(k)

si BUSCAR(k) entonces

 devolver

// k ya estaba en el árbol

en otro caso

$(I, D) \leftarrow \text{SEPARAR}(k)$

$x \leftarrow$ nuevo nodo con valor k

$x.\text{izq} \leftarrow I$, $x.\text{der} \leftarrow D$, $\text{raíz} \leftarrow x$

BUSCAR(k) ya dejó al predecesor o sucesor de k en la raíz — SEPARAR lo parte en dos, y x se cuelga encima. **Ningún** recorrido adicional del árbol.

Eliminar, en pocas líneas



Algoritmo 9: ELIMINAR(k)

si $\neg$ BUSCAR(k) entonces

 devolver // k no está en el árbol

$r \leftarrow \text{raiz}$

$\text{raiz} \leftarrow \text{UNIR}(r.\text{izq}, r.\text{der})$

Eliminar, en pocas líneas



Algoritmo 10: ELIMINAR(k)

si $\neg \text{BUSCAR}(k)$ entonces

 devolver // k no está en el árbol

$r \leftarrow \text{raiz}$

$\text{raiz} \leftarrow \text{UNIR}(r.\text{izq}, r.\text{der})$

BUSCAR(k) deja a k **en la raíz**; sus dos hijos ya cumplen la precondition de UNIR (todo el subárbol izquierdo es menor que todo el derecho) — UNIR hace el resto.

Un mismo costo para las cinco



Contando llamadas a SPLAY

Un mismo costo para las cinco



Contando llamadas a SPLAY

- ▶ BUSCAR, INSERTAR: 1 llamada.

Un mismo costo para las cinco



Contando llamadas a SPLAY

- ▶ BUSCAR, INSERTAR: 1 llamada.
- ▶ UNIR: 1 llamada (splaya el máximo de A).

Un mismo costo para las cinco



Contando llamadas a SPLAY

- ▶ BUSCAR, INSERTAR: 1 llamada.
- ▶ UNIR: 1 llamada (splaya el máximo de A).
- ▶ ELIMINAR = BUSCAR + UNIR: 2 llamadas.

Un mismo costo para las cinco



Contando llamadas a SPLAY

- ▶ BUSCAR, INSERTAR: 1 llamada.
- ▶ UNIR: 1 llamada (splaya el máximo de A).
- ▶ ELIMINAR = BUSCAR + UNIR: 2 llamadas.
- ▶ SEPARAR: ninguna propia — reutiliza la que ya hizo BUSCAR.

Un mismo costo para las cinco



Contando llamadas a SPLAY

- ▶ BUSCAR, INSERTAR: 1 llamada.
- ▶ UNIR: 1 llamada (splaya el máximo de A).
- ▶ ELIMINAR = BUSCAR + UNIR: 2 llamadas.
- ▶ SEPARAR: ninguna propia — reutiliza la que ya hizo BUSCAR.

La consecuencia

Cada operación cuesta, amortizado, a lo más el **doble** de lo que cuesta un SPLAY — así que todo lo que probemos sobre el costo de SPLAY se hereda automáticamente a las cinco. Vamos a probar exactamente eso.

El método del potencial

Peso, tamaño y rango



Definiciones (solo para el análisis, no para el código)

Peso, tamaño y rango



Definiciones (solo para el análisis, no para el código)

- ▶ w_x : un peso positivo arbitrario por nodo (empezaremos con $w_x = 1$ para todos).

Peso, tamaño y rango



Definiciones (solo para el análisis, no para el código)

- ▶ w_x : un peso positivo arbitrario por nodo (empezaremos con $w_x = 1$ para todos).
- ▶ $s(x)$: la suma de los pesos en el subárbol de x (su “tamaño”).

Peso, tamaño y rango



Definiciones (solo para el análisis, no para el código)

- ▶ w_x : un peso positivo arbitrario por nodo (empezaremos con $w_x = 1$ para todos).
- ▶ $s(x)$: la suma de los pesos en el subárbol de x (su “tamaño”).
- ▶ $r(x) = \lg s(x)$: el **rango** de x .

Peso, tamaño y rango



Definiciones (solo para el análisis, no para el código)

- ▶ w_x : un peso positivo arbitrario por nodo (empezaremos con $w_x = 1$ para todos).
- ▶ $s(x)$: la suma de los pesos en el subárbol de x (su “tamaño”).
- ▶ $r(x) = \lg s(x)$: el **rango** de x .

El potencial

$$\Phi = \sum_x r(x)$$

sobre todos los nodos del árbol — intuitivamente, cuánto “trabajo diferido” hay guardado en la forma del árbol.

El Lema de Acceso



¡Piénsalo!

Si pudiéramos acotar el costo amortizado de `SPLAY` en términos de **cuánto cambia el rango de x** , ¿qué forma tendría esa cota?

El Lema de Acceso



¡Piénsalo!

Si pudiéramos acotar el costo amortizado de `SPLAY` en términos de **cuánto cambia el rango de x** , ¿qué forma tendría esa cota?

Lema de Acceso (Sleator & Tarjan, 1985)

El costo amortizado de `SPLAY( $x$ )`, en un árbol con raíz t , es a lo más

$$3(r(t) - r(x)) + 1.$$

El Lema de Acceso



¡Piénsalo!

Si pudiéramos acotar el costo amortizado de `SPLAY` en términos de **cuánto cambia el rango de x** , ¿qué forma tendría esa cota?

Lema de Acceso (Sleator & Tarjan, 1985)

El costo amortizado de `SPLAY( $x$ )`, en un árbol con raíz t , es a lo más

$$3(r(t) - r(x)) + 1.$$

Como x termina siendo la raíz, su rango final es $r(t)$ — así que esta cota dice, esencialmente, “el costo amortizado es proporcional a cuánto **sube** el rango de x ”.

La prueba

Estrategia



Analiza un paso a la vez

SPLAY es una secuencia de pasos Zig-Zig/Zig-Zag (y un Zig final). Si mostramos que **cada paso** cuesta, amortizado, a lo más $3(r'(x) - r(x))$ (con r' el rango **después** de ese paso), la suma telescopa: los rangos intermedios se cancelan, dejando $3(r(t) - r(x))$ — exactamente el Lema de Acceso.

Zig-Zig: qué cambia



Antes y después

Costo real = 2 (dos rotaciones). Solo x, y, z cambian de rango — todo lo demás (los subárboles A, B, C, D) queda igual.

Zig-Zig: qué cambia



Antes y después

Costo real = 2 (dos rotaciones). Solo x, y, z cambian de rango — todo lo demás (los subárboles A, B, C, D) queda igual.

¡Piénsalo!

Después del Zig-Zig, x ocupa exactamente el lugar (y por lo tanto el subárbol completo) que antes ocupaba z . ¿Qué te dice eso sobre $r'(x)$ comparado con $r(z)$?

Zig-Zig: qué cambia



Respuesta: $r'(x) = r(z)$

El conjunto de nodos bajo x después es **exactamente** el mismo que estaba bajo z antes — misma suma de pesos, mismo rango. Además: $r'(x) \geq r(x)$ (x solo puede crecer) y $r'(z) \leq r(z)$ (z solo puede achicarse).

Zig-Zig: acotando el costo amortizado



Costo amortizado = costo real + $\Delta\Phi$

$$2 + (r'(x) + r'(y) + r'(z)) - (r(x) + r(y) + r(z))$$

Zig-Zig: acotando el costo amortizado



Costo amortizado = costo real + $\Delta\Phi$

$$2 + (r'(x) + r'(y) + r'(z)) - (r(x) + r(y) + r(z))$$

Sustituyendo $r'(x) = r(z)$, y usando $r'(y) \leq r'(x)$ y $r(x) \leq r(y)$ (ambas por contención de subárboles):

$$\leq 2 + (r'(z) - r(x)) + (r'(x) - r(x)).$$

Zig-Zig: acotando el costo amortizado



Costo amortizado = costo real + $\Delta\Phi$

$$2 + (r'(x) + r'(y) + r'(z)) - (r(x) + r(y) + r(z))$$

Sustituyendo $r'(x) = r(z)$, y usando $r'(y) \leq r'(x)$ y $r(x) \leq r(y)$ (ambas por contención de subárboles):

$$\leq 2 + (r'(z) - r(x)) + (r'(x) - r(x)).$$

Falta un último paso

Queremos que esto sea $\leq 3(r'(x) - r(x))$ — es decir, necesitamos $2 + r'(z) - r(x) \leq 2(r'(x) - r(x))$. ¿De dónde sale esa desigualdad?

El lema de los logaritmos



Lema (consecuencia de AM-GM)

Si $a, b > 0$ y $a + b \leq c$, entonces

$$\lg a + \lg b \leq 2 \lg c - 2.$$

El lema de los logaritmos



Lema (consecuencia de AM-GM)

Si $a, b > 0$ y $a + b \leq c$, entonces

$$\lg a + \lg b \leq 2 \lg c - 2.$$

Por AM-GM, $ab \leq \left(\frac{a+b}{2}\right)^2 \leq \frac{c^2}{4}$; tomando $\lg$ de ambos lados se obtiene el resultado. El máximo se alcanza cuando $a = b = c/2$.

El lema de los logaritmos



Aplicándolo

Con $a = s'(z)$, $b = s(x)$, $c = s'(x)$ — ya que los subárboles de z (después) y x (antes) son disjuntos y ambos caben dentro del subárbol de x (después) — se obtiene

$$r'(z) + r(x) - 2r'(x) \leq -2,$$

que es exactamente $2 + r'(z) - r(x) \leq 2(r'(x) - r(x))$.

Cerrando Zig-Zig (y Zig-Zag, por simetría)



Conclusión

El costo amortizado de un paso Zig-Zig es

$$\leq 2 + (r'(x) - r(x)) + (r'(x) - r(x)) \cdot 2/2 \leq 3(r'(x) - r(x)).$$

Cerrando Zig-Zig (y Zig-Zag, por simetría)



Conclusión

El costo amortizado de un paso Zig-Zig es

$$\leq 2 + (r'(x) - r(x)) + (r'(x) - r(x)) \cdot 2/2 \leq 3(r'(x) - r(x)).$$

Zig-Zag se demuestra con el mismo esquema (misma desigualdad de logaritmos, aplicada a la configuración reflejada) — no vamos a repetir el álgebra completa, pero el resultado es idéntico: $\leq 3(r'(x) - r(x))$.

El caso base, Zig



El caso base, Zig

Una sola rotación: costo amortizado

$$= 1 + (r'(x) - r(x)) \leq 3(r'(x) - r(x)) + 1.$$

Sumando todo



Telescopando

SPLAY(x) es una cadena de pasos Zig-Zig/Zig-Zag, terminando quizás en un Zig. Sumando el costo amortizado de cada paso, los rangos intermedios se cancelan:

$$\sum \leq 3(r_{\text{final}}(x) - r_{\text{inicial}}(x)) + 1 = 3(r(t) - r(x)) + 1.$$

Sumando todo



Telescopando

SPLAY(x) es una cadena de pasos Zig-Zig/Zig-Zag, terminando quizás en un Zig. Sumando el costo amortizado de cada paso, los rangos intermedios se cancelan:

$$\sum \leq 3(r_{\text{final}}(x) - r_{\text{inicial}}(x)) + 1 = 3(r(t) - r(x)) + 1.$$

Esto es exactamente el Lema de Acceso — demostrado.

Consecuencias

$O(\lg n)$ amortizado



Con pesos uniformes

Si $w_x = 1$ para todo nodo, $s(x)$ es simplemente el número de nodos en el subárbol de x , y $r(\text{raíz}) = \lg n$.

$O(\lg n)$ amortizado



Con pesos uniformes

Si $w_x = 1$ para todo nodo, $s(x)$ es simplemente el número de nodos en el subárbol de x , y $r(\text{raíz}) = \lg n$.

Por el Lema de Acceso: amortizado $\leq 3(\lg n - r(x)) + 1 = O(\lg n)$, para **cualquier** x . Esto confirma lo que anunciamos la clase pasada.

Algo más fuerte: optimalidad estática



¡Piénsalo!

El Lema de Acceso vale para **cualquier** elección de pesos w_x , no solo $w_x = 1$. ¿Qué pasaría si eligiéramos w_x proporcional a **qué tan seguido** se busca x ?

Algo más fuerte: optimalidad estática



¡Piénsalo!

El Lema de Acceso vale para **cualquier** elección de pesos w_x , no solo $w_x = 1$. ¿Qué pasaría si eligiéramos w_x proporcional a **qué tan seguido** se busca x ?

Respuesta: la cota de entropía

Con $w_x \propto p_x$ (la frecuencia de acceso de x), el Lema de Acceso da exactamente $O\left(\sum_x p_x \lg \frac{1}{p_x}\right)$ amortizado — la propiedad de **cota de entropía** que vimos la clase pasada, ahora demostrada.

Cierre de la sesión



- ▶ SPLAY sube x a la raíz vía Zig (caso base), Zig-Zig y Zig-Zag (que balancean el árbol de paso, a diferencia de rotaciones “ingenuas” repetidas).
- ▶ BUSCAR, INSERTAR y ELIMINAR se construyen con SPLAY más SEPARAR/UNIR — cada una cuesta, a lo más, un par de SPLAY.
- ▶ El potencial $\Phi = \sum_x r(x)$, con $r(x) = \lg s(x)$, mide el “trabajo diferido” guardado en la forma del árbol.
- ▶ El **Lema de Acceso** ($3(r(t) - r(x)) + 1$ amortizado) se prueba paso a paso: cada Zig-Zig/Zig-Zag cuesta amortizado $\leq 3(r'(x) - r(x))$, vía el lema de los logaritmos (AM-GM); sumando, telescopa.
- ▶ Con pesos uniformes, esto da $O(\lg n)$ amortizado; con pesos proporcionales a la frecuencia, da la cota de entropía — **un solo lema** prueba ambas propiedades.

Logros de la sesión



Logros de la sesión



- ▶ Sabemos ejecutar SPLAY completo (Zig, Zig-Zig, Zig-Zag).

Logros de la sesión



- ▶ Sabemos ejecutar SPLAY completo (Zig, Zig-Zig, Zig-Zag).
- ▶ Sabemos construir BUSCAR, INSERTAR y ELIMINAR a partir de SPLAY (vía SEPARAR/UNIR).

Logros de la sesión



- ▶ Sabemos ejecutar SPLAY completo (Zig, Zig-Zig, Zig-Zag).
- ▶ Sabemos construir BUSCAR, INSERTAR y ELIMINAR a partir de SPLAY (vía SEPARAR/UNIR).
- ▶ Sabemos definir el potencial de un Splay Tree, y enunciar el Lema de Acceso.

Logros de la sesión



- ▶ Sabemos ejecutar SPLAY completo (Zig, Zig-Zig, Zig-Zag).
- ▶ Sabemos construir BUSCAR, INSERTAR y ELIMINAR a partir de SPLAY (vía SEPARAR/UNIR).
- ▶ Sabemos definir el potencial de un Splay Tree, y enunciar el Lema de Acceso.
- ▶ Sabemos demostrar el Lema de Acceso paso a paso, incluyendo el lema de los logaritmos que lo cierra.

Logros de la sesión



- ▶ Sabemos ejecutar SPLAY completo (Zig, Zig-Zig, Zig-Zag).
- ▶ Sabemos construir BUSCAR, INSERTAR y ELIMINAR a partir de SPLAY (vía SEPARAR/UNIR).
- ▶ Sabemos definir el potencial de un Splay Tree, y enunciar el Lema de Acceso.
- ▶ Sabemos demostrar el Lema de Acceso paso a paso, incluyendo el lema de los logaritmos que lo cierra.
- ▶ Sabemos derivar tanto $O(\lg n)$ amortizado como la cota de entropía a partir del mismo lema.

Gracias

Referencias



Bibliografía citada

- ▶ Sleator, D. D., & Tarjan, R. E. (1985). Self-adjusting binary search trees. *Journal of the ACM*, 32(3).
- ▶ Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. <https://courses.csail.mit.edu/6.851/spring21/>.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.

Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.